Docker Lab: Your First Image

Run a container, build an image, understand layers, make it safer and share it

Duration: about 2 h **Environment:** Docker Desktop or Docker Engine ≥ 24, a terminal, a text editor, `curl`, and a GitLab account. Beginners welcome: no prior Docker experience is needed.

1 Your mission

In the course you saw what a container is. In this lab you use one, then build your own image from the small greeting app of Labs 1 and 2, look inside it, make it safer, and publish it. You work alone or in a pair.

#	Skill	Part	Why it matters
1	Run and inspect containers	1	Everything else builds on these few commands
2	Build an image from a `Dockerfile`	2	The app and its environment travel together
3	Understand layers and cache	3	Faster builds and smaller images
4	Run as non-root and scan	4	A container running as root makes an attack worse
5	Tag and push to a registry	5	An image is useful only if others can pull it

Work in your `devsecops-lab` project and keep the commit convention: `type(scope): summary (#issue)`. Create two issues: #**11** Add a Dockerfile, #**12** Run the container as non-root.

2 Setup (10 min)

Check that Docker works:

```
docker --version
docker run hello-world
```

Read the message printed by `hello-world`: Docker explains what it just did.

Q1. The image `hello-world` was not on your machine. Describe in your own words the steps Docker performed to show you this message.

3 Part 1: Run existing containers (20 min)

3.1 Start a web server

```
docker run -d --name web -p 8080:80 nginx:1.27
```

`-d` runs it in the background, `-name` gives it a name, and `-p 8080:80` publishes the container's port 80 on your port 8080. Open `http://localhost:8080`: you see the nginx welcome page.

3.2 Look around

```
docker ps                       # running containers
docker logs web                 # what nginx printed
docker exec -it web sh          # open a shell inside the container
ls /usr/share/nginx/html        # the page you just saw
exit
```

3.3 Stop and remove

```
docker stop web
docker ps -a                    # stopped containers are still listed
docker rm web
docker images                   # is the nginx image still there?
```

Q2. After `docker rm web`, the `nginx` image is still listed. What is the difference between an image and a container?

4 Part 2: Build your first image (30 min)

4.1 Prepare the project

Create the branch `feature/11-dockerfile` in your `devsecops-lab` project. If you do not have it, create a folder with this `app.py`:

```
def greet(name):
    return "Hello, " + name + "!"
```

Add a tiny web server in `server.py`. It answers every request with the greeting:

```
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import urlparse, parse_qs

from app import greet


class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        name = parse_qs(urlparse(self.path).query).get("name", ["world"])[0]
        body = greet(name).encode()
        self.send_response(200)
        self.send_header("Content-Type", "text/plain; charset=utf-8")
        self.end_headers()
        self.wfile.write(body)


if __name__ == "__main__":
    HTTPServer(("0.0.0.0", 8000), Handler).serve_forever()
```

Test it without Docker first: run `python3 server.py`, then in another terminal `curl "http://localhost:8000/?name=Ana"`. You should see `Hello, Ana!`. Stop the server with Ctrl+C.

4.2 Write the Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py server.py ./
EXPOSE 8000
CMD ["python", "server.py"]
```

Each line is an instruction: start from a small Python image, work in `/app`, copy the two files, document the port, and say what to run.

4.3 Build and run

```
docker build -t greet:1.0 .
docker run -d --name greet -p 8000:8000 greet:1.0
curl "http://localhost:8000/?name=Ana"
docker logs greet
```

`-t greet:1.0` gives the image a name and a version. You should see `Hello, Ana!`, now served from inside a container. Commit your files with `feat(docker): add Dockerfile (#11)`.

5 Part 3: Layers and cache (20 min)

5.1 Look inside the image

```
docker images greet
docker history greet:1.0
```

`docker history` lists the layers: one per instruction, the biggest being the Python base image.

5.2 Change the code and rebuild

Change the greeting in `app.py` (for example `"Welcome, "`), then run the same `docker build` again and read the output carefully. The first layers show `CACHED`: Docker reuses them. Only the layers from `COPY` onwards are rebuilt, because that is where something changed.

5.3 Keep junk and secrets out

Create a file `.dockerignore` next to the `Dockerfile`:

```
.git
.venv
__pycache__/
.env
```

It tells Docker which files must never be sent to the build. This makes builds faster and prevents a local `.env` file from ending up inside an image.

Q3. Why were the first layers reused when you rebuilt? What would happen if you copied your code *before* installing the dependencies of a larger project?

6 Part 4: Make it safer (20 min)

6.1 Who is running your app?

```
docker exec greet whoami
```

The answer is `root`: your small web server runs with full privileges inside the container. If someone exploits it, they start with maximum rights.

6.2 Let a scanner tell you

Trivy, the scanner from Lab 2, can run in a container and read your folder. On Windows PowerShell replace `$PWD` with `${PWD}`.

```
docker run --rm -v "$PWD":/work -w /work aquasec/trivy:0.75.0 config --exit-code 1 --severity HIGH
    ,CRITICAL .
```

It reports a HIGH finding: no `USER` instruction, so the container runs as root. Do not give a scanner access to `/var/run/docker.sock`: that file is equivalent to root access on your machine.

6.3 Fix it

Create the branch `fix/12-non-root` and edit the `Dockerfile`:

```
FROM python:3.12-slim
RUN useradd --create-home appuser
USER appuser
WORKDIR /home/appuser
COPY app.py server.py ./
EXPOSE 8000
CMD ["python", "server.py"]
```

Rebuild with a new version, replace the container, and check:

```
docker build -t greet:1.1 .
docker rm -f greet
docker run -d --name greet -p 8000:8000 greet:1.1
docker exec greet whoami
curl "http://localhost:8000/?name=Ana"
```

You now see `appuser`, and the app still works. Run the Trivy command again: it passes. Commit with `security(docker): run container as non-root (#12)`.

Q4. Why is running a container as root dangerous even though the container is isolated?

7 Part 5: Share your image (15 min)

An image only helps others if they can pull it from a registry. GitLab gives every project one.

7.1 Log in with a token

In GitLab, create a personal access token with the `write_registry` scope. Log in without typing the token in the command line history:

```
echo "<your-token>" | docker login registry.gitlab.com -u <your-username> --password-stdin
```

7.2 Tag and push

```
docker tag greet:1.1 registry.gitlab.com/<owner>/devsecops-lab/greet:1.1
docker push registry.gitlab.com/<owner>/devsecops-lab/greet:1.1
```

The path must be in lowercase. In GitLab, open *Deploy*, then *Container Registry*: your image is listed with its tag. Anyone with access can now run it with `docker run`.

Q5. We pushed the tag `1.1` and never used `latest`. Why is a precise version tag better?

8 Wrap-up: clean up (5 min)

```
docker rm -f greet
docker rmi greet:1.0 greet:1.1
```

```
docker image prune
```

Before you leave, review your `Dockerfile` with this checklist: is the base image version pinned? Does the container run as a non-root user? Is there a `.dockerignore`? Is any secret copied into the image? Does a scan pass?

9 Deliverables and grading (20 pts)

Submit the project link and screenshots of: the output of `hello-world`; the nginx page and `docker ps`; the build output and the `curl` result; `docker history` and the cached rebuild; `whoami` before and after; the Trivy result before and after; the image in the GitLab registry; plus your answers to Q1 to Q5.

Containers run and inspected (Parts 1 and 2 outputs)	4
Image built and running, committed through a branch	4
Layers and cache understood, `.dockerignore` added	3
Non-root fix verified with `whoami` and Trivy	4
Image pushed to the registry	2
Answers Q1 to Q5	3
Total	20

Remember: an image is a template and a container is a running copy; order your Dockerfile to use the cache; never run as root without a reason; tag precise versions.